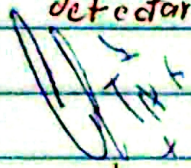
Disciplina de tipos en Programación Lógica y Funcional.

La disciplina de tipos es un conjunto de reglas que determina cómo se clasifican y utilizan los datos dentro de un lenguaje de programación. Un tipo indica qué clase de valor representa una expresión, por ejemplo un número entero un número real un carácter, una cadena de texto o un valor lógico. La disciplina de tipos permite establecer qué operaciones son validas entre los diferentes datos y ayuda a detectar errores durante el desarrollo de un programa.



En la programación funcional, los tipos tienen una relación muy importante con las funciones. Una función puede recibir valores de determinados tipos y producir un resultado de otro tipo. Por ejemplo, una función que suma dos números enteros recibe dos valores enteros y devuelve un entero. Los sistemas de tipos permiten comprobar que los argumentos proporcionados a una función sean compatibles con los tipos que esta espera.

Una característica importante de los lenguajes funcionales es la inferencia de tipos. Esta permite que el propio lenguaje determine el tipo de una expresión a partir de la forma en que se utiliza. Por ejemplo, si una función realiza una operación matemática sobre sus argumentos, el sistema puede deducir que dichos argumentos deben ser compatibles con operaciones numéricas. Esto reduce la cantidad de información que el programador necesita escribir explícitamente.

Escaneado con CamScanner

Elixir

Elixir ayuda a los desarrolladores a escribir código claro y con propósito, centrado en los datos y el dominio. Gracias a la inmutabilidad, la seguridad de la memoria y el sistema de tipos gradual de Elixir, se pueden crear sistemas que se recuperan de fallos y son fáciles de mantener.

Un único código base de Elixir puede escalar verticalmente (en máquinas multinúcleo) y horizontalmente (con un cambio entre nodos) destacando en sistemas orientados a mensajes y en tiempo real para la web. Combinado con proyectos como Numerical Elixir. Elixir escala a través de núcleos, clústeres y GPU

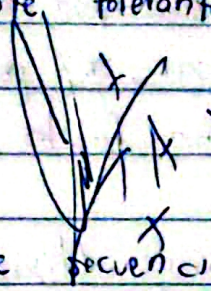
Elixir es utilizado tanto por emprendedores individuales como por empresas de la lista Fortune 500, y brinda soporte a equipos que valoran la satisfacción de los desarrolladores, así como las operaciones a gran escala, en diversos sectores y aplicaciones.

Existen paquetes oficiales y de código abierto (como whatsapp Elixir) diseñado en Elixir para interactuar con la Cloud API de WhatsApp, permitiendo enviar mensajes, plantillas e imágenes automatizadas desde aplicaciones construidas con el framework Phoenix.

Escaneado con CamScanner

Erlang

Erlang es un lenguaje de programación funcional y de propósito general diseñado específicamente para crear sistemas concurrentes, distribuidos y altamente tolerante a fallas.



Características Principales:

- Paradigma funcional: Su parte secuencial se basa en la programación funcional con evaluación estricta e inmutabilidad (las variables tienen asignación única y no cambian de valor) No es un lenguaje de programación lógica (como Prolog) sino estrictamente funcional

Concurrencia nativa: Utiliza procesos ligeros independientes que se comunican mediante paso de mensajes, en lugar de memoria compartida.

Tolerancia a fallos: Famoso por su filosofía "let it crash" (deja que falle), donde los errores se aislan y se reparan de manera automática mediante procesos supervisores.

Uso en tiempo real: Creado originalmente por Ericsson para telecomunicaciones y usado en sistemas masivos como WhatsApp

Escaneado con CamScanner

Funciones de orden superior

Una función de orden superior (higher-order function) es aquella que recibe una o más funciones como argumentos, devuelve otra función como resultado, o realiza ambas acciones a la vez

Este concepto es fundamental en la programación funcional y permite tratar las funciones como ciudadanos de primera clase (valores que se pueden asignar a variables o pasar como datos.

Características principales

Reciben funciones como parámetros: Permiten inyectar lógica personalizada dentro de un flujo genérico (por ejemplo funciones de retrollamada o callbacks).

Devuelven funciones: Permiten generar funciones "al vuelo" o aplicar técnicas como la aplicación parcial y la currificación

Ejemplo comunes en lenguajes de programación

map: Recibe una función y una colección, aplica la función a cada elemento y devuelve una colección con la resultado

filter: Recibe una función de condición (predicado) y una colección, devolviendo solo los elementos que cumplen dicha condición.

reduce o fold: Recibe una función acumuladora y reduce toda una colección a un único valor.

Escaneado con CamScanner

Caracteristica	Prefija	Infija	Postfija
Posición	La función va antes de los argumentos	La función o operador va entre los argumentos	La función iría después del argumento.
Forma general	función argumento 1 argumento 2	argumento 1 operador argumento 2	argumento función
Ejemplo	max 5 8	5 + 8	No es una forma nativa común en Haskell
Resultado de ejemplo	8	13	----
Uso en Haskell	Muy común	Muy común	No se usa directamente como sintaxis estándar de aplicación
Ejemplo típico	sqrt 25	10 * 5	Se puede simular definiendo operadores adecuados
Ventaja	Es clara para llamar funciones normales	Hace las operaciones más fáciles de leer	Puede ser útil para expresar operaciones en estilo encadenado
Importante	Es la forma normal de aplicar funciones	Se utiliza principalmente con operadores	Haskell no tiene aplicación postfija general como otros lenguajes

Escaneado con CamScanner

Operadores en Haskell

Los operadores en Haskell son símbolos que permiten realizar operaciones entre valores, como sumar, restar, multiplicar comparar números o trabajar con condiciones lógicas.

¿Para qué sirven?

Sirven para realizar cálculos matemáticos, comparar datos, evaluar condiciones y manipular dentro de un programa.

Tipos de operadores en Haskell

Tipo	Operadores	Para que sirven
Aritmeticas	+, -, *, /	Realizar operaciones matemática
Comparación	==, /=, >, <, >=, <=	comparar dos valores
Lógicas	&&, ||, not	evaluar condiciones
Listas	++, :	unir listas
Funcionales	$, .	aplicar y combinar funciones

Los operadores en Haskell sirven para realizar operaciones matemáticas, comparar valores, tomar decisiones y trabajar con datos, facilitando la creación de programas mediante expresiones sencillas.

Escaneado con CamScanner

Intervalos en Haskell

Los intervalos en Haskell son una forma de generar listas de valores consecutivos o siguiendo una secuencia determinada, sin necesidad de escribir cada elemento manualmente se representan utilizando dos puntos consecutivos.. dentro de corchetes []

¿Para que sirven?

Sirven para generar secuencias de números, letras y otros valores enumerables de manera rápida y sencilla evitando escribir listas largas

Tipos de intervalos en Haskell

Tipo	Ejemplo	Resultado
Ascendente	[1,..5]	[1,2,3,4,5]
Descendente	[5,4...1]	[5,4,3,2,1]
Numeros pares	[2,4...10]	[2,4,6,8,10]
Números impares	[1,3..9]	[1,3,5,7,9]
Letras	['a'..'e']	[abcde]
Intervalo Infinito	[1...]	1,2,3,4,5...

Los intervalos en Haskell permiten crear listas de valores de forma automática, ordenada y sencilla utilizando la notación... Son útiles para trabajar con secuencias, ciclos y procesamiento de datos.

Escaneado con CamScanner

Evaluación Perezosa en Haskell

La evaluación perezosa (Lazy Evaluation) es una característica de Haskell que consiste en ejecutar o calcular una expresión únicamente cuando el resultado es necesario, en lugar de evaluarla inmediatamente.

¿Para que sirve?

Sirve para optimizar el uso de recursos, evitar calculos innecesarios y permite trabajar con listas infinitas sin tener que calcular todos sus elementos.

Características principales:

- Evaluación bajo demanda: calcula los valores cuando se necesitan
- Listas infinitas: permite trabajar con secuencias sin un límite definido
- Evita calculos innecesarios: las expresiones que no se utilizan pueden permanecer sin evaluar
- Eficiencia: puede ahorrar recursos, aunque también puede consumir más memoria en determinados casos.

Escaneado con CamScanner